The EAOS methodology defines a structured lifecycle for Designing, deploying, and governing enterprise-grade AI systems

Project: Standard Framework Project

3.2 Skill Library

Phase: 3. Capability

[산출물 ID: 3.2 Skill Library.docx]

WBS Task

3.2.1 Agent별 Skill 목록 및 인터페이스 정의
3.2.2 Skill 등록•호출•버전 관리 기준
3.2.3 Skill 의존성 및 권한 매핑

문서 번호	3.2 Skill Library.docx
버전	v1.0
작성일	2026
작성자	KY Song
분류	CONFIDENTIAL
적용 대상	Standard Framework 프로젝트 전체

Table of Contents

1. 개요 3
1.1 Skill 분류 체계 3
1.2 Skill 인터페이스 표준 3
2. Core Skills (공통) 4
2.1 audit_log — 감사 로그 기록 4
2.2 escalate — Escalation 요청 5
2.3 request_approval — 승인 요청 (ApprovalGate) 6
2.4 send_alert — 알림 발송 7
3. Domain Skills — Back-A 전용 8
3.1 generate_api_endpoint — API 엔드포인트 코드 생성 8
3.2 generate_db_migration — DB 마이그레이션 생성 9
3.3 run_static_analysis — 정적 분석 실행 9
4. Domain Skills — Front-A 전용 10
4.1 generate_component — React 컴포넌트 생성 10
4.2 generate_page — Next.js 페이지 생성 11
5. Domain Skills — Test-A 전용 12
5.1 run_simulation — Multi-Agent Simulation 실행 12
5.2 evaluate_kpi — KPI 측정 및 평가 13
6. Domain Skills — Evol-A 전용 14
6.1 analyze_trajectory — Trajectory 분석 14
6.2 propose_improvement — 개선안 제안 14
7. Skill 등록 및 호출 규칙 15
7.1 Skill Registry 15
7.2 Skill 버전 관리 기준 15
8. Skill 전체 목록 요약 16
9. 산출물 체크리스트 16
10. 승인 (Approval) 17

1. 개요

Skill 은 AI Agent 가 특정 작업을 수행하기 위해 호출하는 **캡슐화된 기능 단위**다. Tool 이 외부 시스템(API, DB, 파일)에 접근하는 인터페이스라면, Skill 은 Agent 가 Tool 을 활용해 목적을 달성하는 **실행 레시피**다.

> 💡 Skill = 목적 + Tool 조합 + 실행 순서 + 출력 형식. Agent 는 직접 Tool 을 호출하지 않고 반드시 Skill 을 경유하여 Tool 을 사용한다.

1.1 Skill 분류 체계

```
Skill 분류
├── Core Skills        — 모든 Agent 공통 사용 (로깅, 감사, 에스컬레이션)
├── Domain Skills      — 특정 Agent 전용 (코드 생성, UI 생성, 테스트 실행)
└── Composite Skills   — 2 개 이상의 Skill 을 조합한 복합 Skill
```

1.2 Skill 인터페이스 표준

모든 Skill 은 아래 표준 인터페이스를 준수한다.

```python
class SkillBase:
    """모든 Skill 의 부모 클래스"""
    skill_id: str              # 고유 식별자 (예: "core.audit_log")
    skill_name: str            # 사람이 읽을 수 있는 이름
    version: str               # 버전 (예: "1.0.0")
    allowed_agents: list       # 사용 허용 Agent 목록
    requires_approval: bool    # Human 승인 필요 여부
    audit_required: bool       # 감사 로그 기록 필수 여부

    async def execute(self, input: SkillInput) -> SkillOutput:
        """실행 메서드 — 모든 Skill 이 구현"""
        raise NotImplementedError

    async def validate(self, input: SkillInput) -> bool:
        """실행 전 입력 유효성 검증"""
        raise NotImplementedError

    async def rollback(self, trace_id: str) -> bool:
        """실패 시 롤백 (해당되는 경우)"""
        return False
```

2. Core Skills (공통)

모든 Agent 가 사용 가능한 공통 Skill 이다.

2.1 audit_log — 감사 로그 기록

항목	값
Skill ID	core.audit_log
버전	v1.0.0
허용 Agent	전체
승인 필요	불필요
감사 로그	해당 없음 (자기 자신이 로그)

목적: 모든 Agent 실행 이력을 audit_logs 테이블에 기록한다. 모든 상태 변경 작업 후 반드시 호출해야 한다.

입력 스키마:

```
class AuditLogInput(BaseModel):
    trace_id: UUID
    agent_id: str
    action: str                # 수행한 작업명
    target_type: str         # 대상 엔티티 타입 (예: "user", "agent_run")
    target_id: UUID | None
    before_state: dict | None
    after_state: dict | None
    result: Literal["success", "failure", "pending"]
    metadata: dict = {}
```

출력 스키마:

```
class AuditLogOutput(BaseModel):
    audit_id: UUID
    recorded_at: datetime
    success: bool
```

사용 예시:

```
await skill_registry.execute("core.audit_log", AuditLogInput(
    trace_id=trace_id,
    agent_id="Back-A",
    action="db_schema_change",
    target_type="migration",
    target_id=migration_id,
    before_state=None,
    after_state={"table": "users", "change": "add_column"},
    result="success"
))
```

2.2 escalate — Escalation 요청

항목	값
Skill ID	core.escalate
버전	v1.0.0
허용 Agent	전체
승인 필요	불필요 (에스컬레이션 자체가 승인 요청)
감사 로그	필수

목적: Agent 가 처리 불가능한 상황을 Orch 또는 Human 에게 이관한다.

입력 스키마:

```
class EscalateInput(BaseModel):
    trace_id: UUID
    from_agent: str
    to: Literal["Orch", "Arch", "PM", "Back-H", "Test-H", "Governance-Board"]
    reason: str
    urgency: Literal["low", "medium", "high", "critical"]
    context: dict          # 현재 실행 상태 스냅샷
    suggested_action: str | None
```

출력 스키마:

```
class EscalateOutput(BaseModel):
    escalation_id: UUID
    notified_at: datetime
    channel: str           # "slack" | "email" | "both"
    acknowledged: bool
```

2.3 request_approval — 승인 요청 (ApprovalGate)

항목	값
Skill ID	core.request_approval
버전	v1.0.0
허용 Agent	Orch, Back-A, Evol-A
승인 필요	해당 없음 (승인 요청 Skill)
감사 로그	필수

목적: P1(Pre-approval) 패턴에서 Human 승인을 approval_requests 테이블에 등록하고 알림을 발송한다.

입력 스키마:

```
class RequestApprovalInput(BaseModel):
    trace_id: UUID
    gate_id: str              # "G-03" ~ "G-09"
    requesting_agent: str
    approver_role: str      # "PM" | "Arch" | "Back-H" | "Governance-Board"
    action_description: str
    risk_level: Literal["low", "medium", "high", "critical"]
    deadline_hours: int = 24
    payload: dict             # 승인자에게 보여줄 상세 데이터
```

출력 스키마:

```
class RequestApprovalOutput(BaseModel):
    approval_request_id: UUID
    status: Literal["pending", "approved", "rejected", "timeout"]
    notified_at: datetime
    decision_at: datetime | None
    decision_by: str | None
    rejection_reason: str | None
```

2.4 send_alert — 알림 발송

항목	값
Skill ID	core.send_alert
버전	v1.0.0
허용 Agent	전체
승인 필요	불필요
감사 로그	필수

입력 스키마:

```
class SendAlertInput(BaseModel):
    trace_id: UUID
    channel: Literal["slack", "email", "both"]
    recipient: str
    subject: str
    body: str
    urgency: Literal["low", "medium", "high", "critical"]
    metadata: dict = {}
```

3. Domain Skills — Back-A 전용

3.1 generate_api_endpoint — API 엔드포인트 코드 생성

항목	값
Skill ID	back.generate_api_endpoint
버전	v1.0.0
허용 Agent	Back-A
승인 필요	P3 (Post-review, Back-H 확정)
감사 로그	필수

목적: FastAPI 라우터 · 서비스 · 레포지토리 코드를 도메인 구조에 맞게 생성한다.

입력 스키마:

```
class GenerateApiInput(BaseModel):
    trace_id: UUID
    domain: str                # 예: "users", "agents", "billing"
    endpoint_name: str       # 예: "create_user", "get_agent_run"
    http_method: Literal["GET", "POST", "PUT", "PATCH", "DELETE"]
    request_schema: dict   # Pydantic 필드 정의
    response_schema: dict
    auth_required: bool = True
    audit_log_required: bool = True
    include_test: bool = True
```

출력 스키마:

```
class GenerateApiOutput(BaseModel):
    trace_id: UUID
    files: list[dict]        # [{"path": str, "content": str}]
    review_required: bool = True
    reviewer: str = "Back-H"
    test_file_included: bool
```

생성 파일 목록:

```
domains/{domain}/
├── router.py          ← FastAPI 라우터
├── service.py         ← 비즈니스 로직
├── repository.py      ← DB 쿼리
├── schemas.py         ← Pydantic 스키마
└── tests/
    └── test_{endpoint_name}.py
```

3.2 generate_db_migration — DB 마이그레이션 생성

항목	값
Skill ID	back.generate_db_migration
버전	v1.0.0
허용 Agent	Back-A
승인 필요	**P1 필수** (G-05 Decision Gate)
감사 로그	필수

목적: Alembic 마이그레이션 파일 초안을 생성한다. 실제 DB 반영은 반드시 Arch + Back-H 승인 후 수동 실행한다.

입력 스키마:

```
class GenerateMigrationInput(BaseModel):
    trace_id: UUID
    table_name: str
    change_type: Literal["create_table", "add_column", "alter_column", "drop_column", "add_index"]
    change_description: str
    columns: list[dict]    # 컬럼 정의
    rollback_plan: str     # 롤백 방법 명시 필수
```

> ⚠□ **이 Skill 은 P1 ApprovalGate 없이 실행 불가.** Back-A 는 생성 직후 core.request_approval Skill 을 반드시 호출한다.

3.3 run_static_analysis — 정적 분석 실행

항목	값
Skill ID	back.run_static_analysis
버전	v1.0.0
허용 Agent	Back-A, Test-A
승인 필요	불필요
감사 로그	필수

목적: 생성된 Python 코드에 대해 ruff, mypy, bandit 을 자동 실행하고 결과를 반환한다.

```
class StaticAnalysisInput(BaseModel):
    trace_id: UUID
    file_paths: list[str]
    tools: list[Literal["ruff", "mypy", "bandit"]] = ["ruff", "mypy", "bandit"]

class StaticAnalysisOutput(BaseModel):
    trace_id: UUID
    results: dict          # {"ruff": {...}, "mypy": {...}, "bandit": {...}}
    passed: bool
    issues: list[dict]
```

4. Domain Skills — Front-A 전용

4.1 generate_component — React 컴포넌트 생성

항목	값
Skill ID	front.generate_component
버전	v1.0.0
허용 Agent	Front-A
승인 필요	P3 (Post-review, Front-H 확정)
감사 로그	필수

목적: TypeScript/React 함수형 컴포넌트를 생성한다. shadcn/ui 기반, 디자인 토큰 적용, 접근성 준수가 자동 포함된다.

입력 스키마:

```
class GenerateComponentInput(BaseModel):
    trace_id: UUID
    component_name: str     # PascalCase (예: "ApprovalGate")
    component_type: Literal["ui", "agent", "mobile", "layout"]
    props: list[dict]       # TypeScript props 정의
    description: str
    dark_mode: bool = True
    accessibility: bool = True
    include_storybook: bool = False
```

출력 규칙:

```
components/
├── ui/          → shadcn 래퍼 컴포넌트
├── agents/      → AI Agent 전용 (AgentStatusBadge, ApprovalGate 등)
├── mobile/      → /m/* 전용 모바일 컴포넌트
└── layout/      → 레이아웃 (Sidebar, Header 등)
```

4.2 generate_page — Next.js 페이지 생성

항목	값
Skill ID	front.generate_page
버전	v1.0.0
허용 Agent	Front-A
승인 필요	P3 (Post-review, Front-H 확정)
감사 로그	필수

입력 스키마:

```
class GeneratePageInput(BaseModel):
    trace_id: UUID
    route: str                  # 예: "/dashboard", "/m/approvals"
    page_type: Literal["list", "detail", "form", "dashboard"]
    is_mobile: bool = False
    auth_required: bool = True
    data_fetching: Literal["server", "client", "hybrid"] = "server"
```

5. Domain Skills — Test-A 전용

5.1 run_simulation — Multi-Agent Simulation 실행

항목	값
Skill ID	test.run_simulation
버전	v1.0.0
허용 Agent	Test-A
승인 필요	P1 (Test-H 승인 후 실행)
감사 로그	필수

목적: 승인된 Test Scenario 를 기반으로 Multi-Agent 대화 시뮬레이션을 실행하고 결과를 trajectory_logs 에 기록한다.

입력 스키마:

```
class RunSimulationInput(BaseModel):
    trace_id: UUID
    scenario_id: str
    scenario_type: Literal["happy_path", "role_conflict", "tool_failure", "timeout", "injection"]
    agents_involved: list[str]
    max_turns: int = 20
    record_trajectory: bool = True
```

출력 스키마:

```
class RunSimulationOutput(BaseModel):
    trace_id: UUID
    scenario_id: str
    status: Literal["passed", "failed", "partial"]
    turns_executed: int
    kpi_results: dict
    failure_points: list[dict]
    trajectory_log_id: UUID
```

5.2 evaluate_kpi — KPI 측정 및 평가

항목	값
Skill ID	test.evaluate_kpi
버전	v1.0.0
허용 Agent	Test-A, Monitor-A
승인 필요	불필요
감사 로그	필수

KPI 측정 항목:

KPI	측정 방법	임계값 (Pass 기준)
Task Completion Rate	완료 시나리오 / 전체 시나리오	≥ 90%
Escalation Accuracy	올바른 에스컬레이션 / 전체 에스컬레이션	≥ 95%
Guardrail Violation Rate	위반 건수 / 전체 실행	≤ 0%
Approval Gate Response Time	평균 승인 대기 시간	≤ 24h
Audit Log Coverage	감사 로그 기록 건수 / 전체 실행	= 100%
Prompt Injection Block Rate	차단 건수 / 탐지 건수	= 100%

6. Domain Skills — Evol-A 전용

6.1 analyze_trajectory — Trajectory 분석

항목	값
Skill ID	evol.analyze_trajectory
버전	v1.0.0
허용 Agent	Evol-A
승인 필요	불필요 (분석만, 적용은 별도 승인)
감사 로그	필수

목적: trajectory_logs 데이터를 분석하여 실패 패턴을 식별하고 개선 우선순위를 도출한다.

```
class AnalyzeTrajectoryInput(BaseModel):
    trace_id: UUID
    period_days: int = 7
    filter_agent: str | None = None
    filter_status: Literal["failed", "partial", "all"] = "failed"
    min_occurrence: int = 3    # 패턴 최소 발생 횟수
```

6.2 propose_improvement — 개선안 제안

항목	값
Skill ID	evol.propose_improvement
버전	v1.0.0
허용 Agent	Evol-A
승인 필요	**P1 필수** (G-09 Governance Board)
감사 로그	필수

목적: 분석 결과를 기반으로 System Prompt · Workflow · Tool Config 개선안을 Patch Draft 형태로 생성하고 Governance Board 승인을 요청한다.

```
class ProposeImprovementInput(BaseModel):
    trace_id: UUID
    analysis_id: UUID        # analyze_trajectory 결과 참조
    target: Literal["system_prompt", "workflow", "tool_config", "memory"]
    proposal: str
    expected_kpi_delta: dict   # 예: {"Task Completion Rate": "+3%"}
    regression_test_plan: str
```

---

7. Skill 등록 및 호출 규칙

7.1 Skill Registry

```
# skill_registry.py — 전체 Skill 등록 관리
class SkillRegistry:
    _skills: dict[str, SkillBase] = {}

    @classmethod
    def register(cls, skill: SkillBase):
        cls._skills[skill.skill_id] = skill

    @classmethod
    async def execute(cls, skill_id: str, input: SkillInput) -> SkillOutput:
        skill = cls._skills.get(skill_id)
        if not skill:
            raise SkillNotFoundError(skill_id)

        # 1. Agent 권한 확인
        if input.agent_id not in skill.allowed_agents:
            raise SkillPermissionError(skill_id, input.agent_id)

        # 2. 입력 유효성 검증
        await skill.validate(input)

        # 3. 승인 필요 여부 확인
        if skill.requires_approval and not input.approval_id:
            raise ApprovalRequiredError(skill_id)

        # 4. 실행
        output = await skill.execute(input)

        # 5. 감사 로그 (audit_required인 경우)
        if skill.audit_required:
            await cls.execute("core.audit_log", ...)

        return output
```

7.2 Skill 버전 관리 기준

변경 유형	버전 변경	승인 필요
버그 수정, 오타	Patch (x.x.1)	Arch
입력/출력 스키마 추가	Minor (x.1.0)	Arch + Back-H
Skill 목적·구조 변경	Major (2.0.0)	Governance Board
신규 Skill 추가	Minor (x.1.0)	Arch + 해당 Domain 담당자
Skill 폐기(Deprecation)	Major	Governance Board

8. Skill 전체 목록 요약

Skill ID	이름	허용 Agent	승인 필요
core.audit_log	감사 로그 기록	전체	불필요
core.escalate	Escalation 요청	전체	불필요
core.request_approval	승인 요청	Orch, Back-A, Evol-A	불필요
core.send_alert	알림 발송	전체	불필요
back.generate_api_endpoint	API 엔드포인트 생성	Back-A	P3
back.generate_db_migration	DB 마이그레이션 생성	Back-A	**P1 필수**
back.run_static_analysis	정적 분석 실행	Back-A, Test-A	불필요
front.generate_component	React 컴포넌트 생성	Front-A	P3
front.generate_page	Next.js 페이지 생성	Front-A	P3
test.run_simulation	Multi-Agent Simulation	Test-A	P1
test.evaluate_kpi	KPI 측정	Test-A, Monitor-A	불필요
evol.analyze_trajectory	Trajectory 분석	Evol-A	불필요
evol.propose_improvement	개선안 제안	Evol-A	**P1 필수**

9. 산출물 체크리스트

#	항목	완료
1	Core Skill 4 개 인터페이스 정의 완료	☐
2	Back-A Domain Skill 3 개 정의 완료	☐
3	Front-A Domain Skill 2 개 정의 완료	☐
4	Test-A Domain Skill 2 개 정의 완료	☐
5	Evol-A Domain Skill 2 개 정의 완료	☐
6	Skill Registry 구조 정의 완료	☐
7	Skill 버전 관리 기준 정의 완료	☐
8	Architect 검토 완료	☐
9	Tool Config(3.2.2)와 연동 확인 완료	☐

> ✅ 9 개 항목 전체 완료 → **Phase 3 Skill Library 적용 가능**

10. 승인 (Approval)

구분	내용	서명	날짜
작성자			
검토자 (AI Architect)			
승인자 (Business Owner)			
버전	v1.0		